

II EN RÉSUMÉ POUR L'ORAL (PHRASE CLÉ À DIRE AU PROFESSEUR / JURY)

« C'est un moteur piloté par les métadonnées (Metadata-Driven Architecture) : le code Java ne connaît pas les statuts à l'avance ; il projette dynamiquement le modèle BPMN dans l'interface et dans la base de données. »

1. Zéro Statut Hardcodé : Aucun Enum Java rigide

2. Hot-Reloading : Nouveau cycle de vie sans recompiler

3. Continuité d'Audit : Traçabilité native dans Flowable BDD

I LES 3 RÈGLES DE CONCEPTION FONDAMENTALES DU WORKFLOW

1. Règle de Syntaxe BPMN 2.0 Standard (Intégrité Topologique)

MOTEUR FLOWABLE

- Identifiant fixe de processus** : Le fichier XML doit obligatoirement déclarer `<process id="incidentWorkflow">`. Cela permet à Flowable de versionner automatiquement les déploiements (`VERSION_`) dans `flowable.act_re_procdef` sans rupture de clé.
- Graphe continu sans orphelins** : Le schéma doit posséder un `StartEvent`, au moins une `UserTask` ou un état terminal `EndEvent`, et des `SequenceFlow` directionnels reliant tous les nœuds pour garantir l'avancement déterministe du jeton d'exécution.
- Normalisation automatique** : Le backend Spring Boot valide et normalise l'extension en `.bpmn20.xml` même si le navigateur l'a renommée lors du téléchargement.

2. Règle d'Attribution Dynamique des Couleurs UI (Harmonie Visuelle)

FRONTEND ADAPTATIF

- Distribution modulaire** : Pour que l'interface reste toujours esthétique et lisible sans intervention humaine, le backend applique un algorithme de distribution dynamique calculé par : `colors[i % colors.length]`.
- Génération automatique des badges Tailwind** : Chaque état hérite automatiquement d'une palette cohérente (fond, texte, bordure) partagée entre le Kanban, le Stepper et le graphe React Flow : `1. Rouge` `2. Ambre` `3. Bleu` `4. Émeraude` `5. Ardoise` `6. Pourpre`.
- Zéro retouche CSS** : Un nouvel état (« Inspection Terrain », « En Attente Fournisseur ») est stylisé instantanément dès l'ingestion XML.

3. Règle de Persistance Métier & Préservation d'Histoire

POSTGRESQL MULTI-SCHÉMAS

- Remplacement atomique du workflow** : Lors d'une synchronisation, les nouveaux états remplacent les anciens dans la table métier `incidentflow.workflow_states` et le cache Redis est purgé via `@CacheEvict`.
- Préservation intégrale des incidents passés** : L'historique des incidents déjà créés ou clôturés reste intact et inaltéré grâce au schéma d'audit Flowable (`flowable.act_hi_taskinst`) et à l'historique d'audit applicatif (`incidentflow.incident_history`).
- Soft Foreign Key** : `incidents.process_instance_id` lie l'incident au moteur Flowable sans contrainte SQL `FOREIGN KEY` rigide inter-schémas, garantissant l'indépendance de migration.

I ARCHITECTURE CLASSIQUE RIGIDE VS INCIDENTFLOW V2 METADATA-DRIVEN

CRITÈRE D'ÉVALUATION	APPROCHE TRADITIONNELLE (RIGIDE)	INCIDENTFLOW V2 (METADATA-DRIVEN)
Cycle de Vie & Statuts	✗ Enums Java figés. Recompilation et redémarrage obligatoires.	✓ Modèle BPMN XML ingéré à chaud par Flowable en BDD.
Interface Kanban / UI	✗ Colonnes React codées en dur, désynchronisées du backend.	✓ Colonnes, badges et graphe React Flow projetés du XML.
Traçabilité & Audit	✗ Historique fragile ou perdu lors du renommage d'un statut.	✓ Schéma d'audit Flowable (<code>act_hi_*</code>) infalsifiable et pérenne.

CHECKLIST DÉMONSTRATION DIRECTE DEVANT LE PROFESSEUR

- ☐ 1. Glisser-Déposer le XML : Dans l'onglet *Workflow Config*, importer un nouveau fichier BPMN (ex: avec un statut renommé ou ajouté).
- ☐ 2. Vérifier la BDD : Montrer la requête `SELECT * FROM incidentflow.workflow_states;` : les colonnes ont changé automatiquement !
- ☐ 3. Montrer le Kanban : Ouvrir le Kanban : colonnes et couleurs sont recalculées instantanément sans recharger la page.
- ☐ 4. Conclure avec la phrase clé : Énoncer le pitch : « Moteur piloté par les métadonnées (Metadata-Driven) ».